

Reviewer Pack — Minimal Local Cohomology in Algebraic Geometry and Resolutio...

Entry 98d5c23f6e46 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 03:39:14 UTC

Minimal Local Cohomology in Algebraic Geometry and Resolution Proof Width Correlation

Entry ID: 98d5c23f6e46 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 03:39:14 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Local Cohomology) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

The resolution proof width of a Boolean satisfiability problem is linearly correlated with the minimal local cohomology rank over an algebraically closed field k , such that $w(\varphi) = \Theta(h_1(\varphi))$, where $h_1(\varphi)$ is the minimal local cohomology rank for the algebraic variety associated with the formula φ .

Rationale (proposer's reasoning):

Local cohomology in algebraic geometry has been used to study properties of varieties. If a direct connection between local cohomology and resolution complexity can be established, it may reveal new insights into the structure of satisfiability problems and potentially lead to more efficient algorithms for solving them.

Taxonomy category: Algebraic_Geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 863dab4da79a0be9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if Pearson correlation coefficient (r) between resolution proof width and minimal local cohomology rank exceeds 0.7 across all instances, with no seed producing a correlation below 0.6.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'minimal local cohomology' AND 'resolution proof width' algebraic geometry - 'Boolean satisfiability' AND 'algebraic variety' AND 'local cohomology rank' - correlation 'resolution proof complexity' 'minimal local cohomology'

Top relevant hits considered: - [http://arxiv.org/abs/0707.3937v1] Cohomology theories for homotopy algebras and noncommutative geometry - [http://arxiv.org/abs/1209.3595v2] Noncommutative complex differential geometry - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/2005.11229v3] Cohomology of algebraic varieties over non-archimedean fields - [http://arxiv.org/abs/1109.2049v1] Structure-Based Local Search Heuristics for Circuit-Level Boolean Satisfiability - [http://arxiv.org/abs/1705.03356v2] Growth in varieties of multiplier algebras and Groebner bases in operads - [http://arxiv.org/abs/0909.3441v3] Introduction into "Local Correlation Modelling" - [http://arxiv.org/abs/math/0406355v1] p-torsion elements in local cohomology modules. II

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                continue
            for j in range(n):
                A[i][j] /= A[i][i]
            for k in range(m):
                if k != i and A[k][i] != 0:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
```

```

C = [[0] * p for _ in range(m)]
for i in range(m):
    for j in range(p):
        for k in range(n):
            C[i][j] += A[i][k] * B[k][j]
return C

def determinant(A):
    if len(A) == 1:
        return A[0][0]
    det = 0
    for c in range(len(A)):
        det += ((-1) ** c) * A[0][c] * determinant([row[:c] + row[c+1:] for row in A[1:]])
    return det

def local_cohomology_rank(n):
    # Example encoding of a Boolean formula to an algebraic variety
    # This is a placeholder implementation and should be replaced with actual computation
    # For simplicity, we use the rank of a random matrix as a proxy for local cohomology rank
    A = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    return abs(determinant(gaussian_elimination(A)))

def resolution_width(n):
    # Example encoding of a Boolean formula to its resolution width
    # This is a placeholder implementation and should be replaced with actual computation
    # For simplicity, we use the rank of a random matrix as a proxy for resolution width
    A = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    return abs(determinant(gaussian_elimination(A)))

n_max = 40
instances_tested = 30
metric_values = []
conjecture_holds = True
counterexample = ""

for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(instances_tested // len([5, 10, 15, 20, 30, 40])):
        h_1 = local_cohomology_rank(n)
        w_phi = resolution_width(n)
        metric_values.append(w_phi / h_1)

if len(metric_values) == 0:
    return {
        "metric_name": "resolution_width_over_local_cohomology",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean = sum(metric_values) / len(metric_values)
std_dev = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
correlation_coefficient = sum((metric_values[i] - mean) * (i + 1 - mean) for i in range(len(metric_values)))

if correlation_coefficient < 0.6:

```

```

conjecture_holds = False
counterexample = f"Correlation coefficient {correlation_coefficient} below threshold"

return {
    "metric_name": "resolution_width_over_local_cohomology",
    "metric_value": mean,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys

    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_metric_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
    std_dev_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if all("counterexample" in result and result["counterexample"] != "" for result in results):
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={seeds[0]}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_dev_metric_value} support_fraction={support_fraction}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc58b3d1.py", line 114, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc58b3d1.py", line 76, in run_trial
    h_1 = local_cohomology_rank(n)
          ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc58b3d1.py", line 59, in local_cohomology_rank
    return abs(determinant(gaussian_elimination(A)))
                  ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc58b3d1.py", line 25, in gaussian_elimination
    A[i], A[max_row] = A[max_row], A[i]
                        ~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Pearson correlation coefficient required to support or falsify the conjecture. | next: Investigate and fix the crash in the test code to allow for proper evaluation of the Pearson correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19999
2	preregistration	ollama_remote	glm4:latest	0	0	12174
3	novelty	ollama_remote	glm4:latest	0	0	8747
4	novelty	ollama_remote	glm4:latest	0	0	10622
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20239
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14242
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12495
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14274
9	judge	ollama_remote	glm4:latest	0	0	13523

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 126314 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/98d5c23f6e46.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/98d5c23f6e46.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/98d5c23f6e46.tar.gz (if generated)